

Dokumentacja Wewnętrzna Systemu

Spis Treści

1. Przegląd Systemu
 2. Architektura Aplikacji
 3. Komponenty Systemu
 4. Przepływ Danych
 5. Konfiguracja i Środowisko
 6. Integracje Zewnętrzne
 7. Bezpieczeństwo
 8. Zarządzanie Stanem
 9. Optymalizacja i Performance
-

Przegląd Systemu

Cel Aplikacji

Aplikacja to zaawansowany system generowania interfejsów użytkownika oparty na sztucznej inteligencji. System umożliwia użytkownikom tworzenie komponentów React poprzez rozmowę w języku naturalnym, oferując wizualizację generowanych komponentów w czasie rzeczywistym.

Główne Funkcjonalności

- **Generowanie UI poprzez chat:** Użytkownik opisuje potrzebny interfejs, a LLM generuje kod React
- **Live Preview:** Natychmiastowa wizualizacja wygenerowanych komponentów
- **Historia konwersacji:** Automatyczne zapisywanie i zarządzanie historią czatów
- **Wsparcie dla obrazów:** Możliwość przesyłania obrazów referencyjnych
- **Rozpoznawanie mowy:** Integracja z Web Speech API dla wprowadzania głosowego
- **Biblioteka komponentów:** Pełna integracja z shadcn/ui (25+ komponentów)

Stack Technologiczny

- **Framework:** Next.js 16.0.4 (App Router)
- **React:** 19.2.0
- **TypeScript:** 5.x
- **Styling:** Tailwind CSS 3.4.0
- **UI Components:** shadcn/ui (Radix UI primitives)
- **LLM Integration:** Groq API (Llama 4 Maverick)
- **Parser:** Babel Parser 7.22.15

- **Wykresy:** Recharts 2.15.4
- **Formularze:** React Hook Form 7.68.0 + Zod 4.1.13

Architektura Aplikacji

Struktura Projektu

```

/
  app/                                # Next.js App Router
    page.tsx                          # Strona główna - orkiestrator aplikacji
    layout.tsx                        # Root layout z metadanymi
    globals.css                       # Style globalne, zmienne CSS
    api/
      chat/
        route.ts                      # API endpoint dla LLM
  components/                         # Komponenty React
    ChatInterface.tsx                # Główny interfejs czatu
    PreviewPanel.tsx                 # Panel podglądu komponentów
    HistoryModal.tsx                 # Modal historii konwersacji
    ui/                              # Komponenty shadcn/ui
      button.tsx
      input.tsx
      dialog.tsx
      [25+ innych komponentów]
  lib/                               # Utilities i logika biznesowa
    chat-history.ts                  # Zarządzanie historią czatów
    utils.ts                         # Pomocnicze funkcje (cn, etc.)
    tailwind-safelist.tsx           # Safelist klas Tailwind
  types/                             # Definicje TypeScript
    chat.ts                          # Typy dla systemu czatu
  public/                            # Zasoby statyczne
  tailwind.config.mjs                # Konfiguracja Tailwind CSS
  tsconfig.json                      # Konfiguracja TypeScript
  package.json                       # Zależności projektu

```

Wzorce Architektoniczne

1. Architektura klient-serwer

- **Komponenty serwerowe:** Minimalne użycie (layout.tsx)
- **Komponenty klienckie:** Większość UI (“use client” directive)
- **Ścieżki API:** Endpoint /api/chat obsługujący komunikację z LLM

2. Architektura komponentów

- **Atomic Design:** Komponenty od atomic (Button) do organisms (Chat-Interface)
- **Composition Pattern:** Komponenty złożone z mniejszych części
- **Wrapper Components:** Interactive wrappers dla stateful components

3. Wzorzec State Management

- **Local State:** useState dla prostych stanów
- **LocalStorage:** Persistencja historii konwersacji

Komponenty Systemu

1. Strona Główna (app/page.tsx)

Odpowiedzialność: Orkiestrator całej aplikacji, zarządzanie globalnym stanem

Stan Komponentu:

```
const [messages, setMessages] = useState<Message[]>([]); // Historia wiadomości
const [currentChatId, setCurrentChatId] = useState<string>(""); // ID aktywnego czatu
const [chatHistory, setChatHistory] = useState<Chat[]>([]); // Lista wszystkich czatów
const [historyModalOpen, setHistoryModalOpen] = useState(false); // Stan modalu historii
```

Kluczowe Funkcje:

- handleNewChat(): Inicjalizacja nowej konwersacji z unikalnym ID
- handleLoadChat(chat: Chat): Ładowanie historycznej konwersacji
- handleDeleteChat(chatId: string): Usuwanie konwersacji
- Auto-save mechanism (useEffect) - automatyczne zapisywanie po każdej zmianie

Przepływ Danych:

```
[page.tsx]
  > [ChatInterface] (messages, setMessages, callbacks)
  > [PreviewPanel] (messages)
  > [HistoryModal] (chatHistory, callbacks)
```

2. ChatInterface (components/ChatInterface.tsx)

Odpowiedzialność: Interfejs użytkownika do komunikacji z LLM

Stan Lokalny:

```
const [inputValue, setInputValue] = useState(""); // Treść inputa
const [selectedImages, setSelectedImages] = useState<string[]>([]); // Wybrane obrazy
const [isRecording, setIsRecording] = useState(false); // Status nagrywania
const recognitionRef = useRef<SpeechRecognition | null>(null); // Ref do Web Speech API
```

Kluczowe Funkcjonalności:

1. Obsługa Wiadomości:

```
const handleSendMessage = async () => {  
  // 1. Dodaj wiadomość użytkownika  
  const newMessage: Message = { id, role: "user", content, images };  
  setMessages((prev) => [...prev, newMessage]);  
  
  // 2. Dodaj loading message  
  setMessages((prev) => [...prev, loadingMessage]);  
  
  // 3. Wywołaj API  
  const response = await fetch("/api/chat", {  
    method: "POST",  
    body: JSON.stringify({ messages }),  
  });  
  
  // 4. Zastąp loading message odpowiedzią  
  const data = await response.json();  
  setMessages((prev) =>  
    prev.map((msg) => (msg.id === loadingId ? assistantMessage : msg))  
  );  
};
```

2. Obsługa Obrazów:

- Konwersja plików do Base64 (FileReader API)
- Wyświetlanie miniatur z opcją usunięcia
- Przesyłanie do LLM jako image_url

3. Rozpoznawanie Mowy:

```
// Inicjalizacja Web Speech API  
if ("SpeechRecognition" in window || "webkitSpeechRecognition" in window) {  
  const SpeechRecognition =  
    window.SpeechRecognition || window.webkitSpeechRecognition;  
  recognitionRef.current = new SpeechRecognition();  
  recognitionRef.current.continuous = false;  
  recognitionRef.current.lang = "pl-PL";  
  
  recognitionRef.current.onresult = (event) => {  
    const transcript = event.results[0][0].transcript;  
    setInputValue((prev) => prev + " " + transcript);  
  };  
}
```

Renderowanie Wiadomości:

- Różnicowanie stylów dla user/assistant

- Markdown support dla odpowiedzi LLM
- Wyświetlanie załączonych obrazów
- Code highlighting dla bloków kodu

3. PreviewPanel (components/PreviewPanel.tsx)

Odpowiedzialność: Renderowanie wygenerowanych komponentów w czasie rzeczywistym

Architektura Parsowania:

1. Ekstrakcja Kodu:

```
const extractJSX = (content: string) => {
  const codeBlockMatch = content.match(/```(?:tsx|jsx)?\n([\s\S]*?)\n```/);
  return codeBlockMatch ? codeBlockMatch[1].trim() : null;
};
```

2. Parsowanie JSX:

```
// Używa @babel/parser
const ast = parse(jsxCODE, {
  sourceType: "module",
  plugins: ["jsx", "typescript"],
});
```

3. Konwersja:

```
function jsxToTree(node: any): ComponentTree {
  return {
    type: "element",
    name: getJSXName(node.openingElement.name),
    props: extractProps(node.openingElement.attributes),
    children: processChildren(node.children),
  };
}
```

4. Renderowanie:

```
const renderNode = (node: ComponentTree): React.ReactNode => {
  const Component = componentMap[node.name];
  return (
    <Component {...node.props}>
      {node.children.map((child) => renderNode(child))}
    </Component>
  );
};
```

Interactive Wrappers:

Komponenty wymagające stanu są owijane we “wrappery”:

```

function InteractiveTabs({
  children,
  defaultValue,
  value: propValue,
  ...props
}: any) {
  const initialValue = propValue || defaultValue || "tab1";
  const [value, setValue] = useState(initialValue);

  return (
    <Tabs
      value={value}
      onChange={setValue}
      defaultValue={initialValue}
      {...props}
    >
      {children}
    </Tabs>
  );
}

```

Dostępne “wrappery”:

- InteractiveTabs - zarządzanie aktywną zakładką
- InteractiveAccordion - zarządzanie rozwinięciem
- InteractiveDialog - kontrola otwarcia modala
- InteractivePopover - kontrola otwarcia popovera
- InteractiveCalendar - wybór daty
- InteractiveTooltip - wyświetlanie tooltipa
- InteractiveChart - dostarczenie domyślnej konfiguracji

Mapa komponentów:

```

const componentMap: Record<string, any> = {
  Button,
  Input,
  Label,
  Checkbox,
  Tabs: InteractiveTabs,
  TabsList,
  TabsTrigger,
  TabsContent,
  Accordion: InteractiveAccordion,
  AccordionItem,
  AccordionTrigger,
  AccordionContent,
  Dialog: InteractiveDialog,
  Popover: InteractivePopover,
}

```

```

    Calendar: InteractiveCalendar,
    Chart: InteractiveChart,
    // ... pozostałe 20+ komponentów
  };

```

4. Ścieżka API (app/api/chat/route.ts)

Odpowiedzialność: Komunikacja z Groq API, parsowanie JSX

Proces Obsługi Zapytania:

```

export async function POST(request: NextRequest) {
  // 1. Walidacja API Key
  if (!process.env.GROQ_API_KEY) {
    return NextResponse.json(
      { error: "GROQ_API_KEY is not configured" },
      { status: 500 }
    );
  }

  // 2. Przygotowanie wiadomości z system prompt
  const groqMessages = [
    { role: "system", content: SYSTEM_PROMPT },
    ...messages.map(transformMessage),
  ];

  // 3. Wywołanie Groq API
  const response = await fetch(
    "https://api.groq.com/openai/v1/chat/completions",
    {
      method: "POST",
      headers: { Authorization: `Bearer ${process.env.GROQ_API_KEY}` },
      body: JSON.stringify({
        model: "meta-llama/llama-4-maverick-17b-128e-instruct",
        messages: groqMessages,
        max_tokens: 4096,
        temperature: 0.7,
      }),
    },
  );

  // 4. Ekstrakcja i parsowanie JSX
  const assistantMessage = data.choices[0].message.content;
  const jsxCode = extractJSX(assistantMessage);

  if (jsxCode) {
    const tree = parseJSXToTree(jsxCode);
  }
}

```

```

        return NextResponse.json({
            message: assistantMessage,
            tree,
        });
    }

    return NextResponse.json({ message: assistantMessage });
}

```

Prompt - Instrukcje dla LLM:

Kluczowe elementy:

- Lista dostępnych komponentów (25+)
- Wymagany format odpowiedzi (text + code block)
- Zasady użycia inline styles (zamiast className)
- Reguły dla konkretnych komponentów (defaultValue dla Tabs, etc.)
- Wzorce stylowania (flexbox, spacing, colors)

Parsowanie JSX:

```

function literalFromNode(node: any): any {
    if (node.type === "StringLiteral") return node.value;
    if (node.type === "NumericLiteral") return node.value;
    if (node.type === "BooleanLiteral") return node.value;

    // Obsługa ObjectExpression dla style props
    if (node.type === "ObjectExpression") {
        const obj: Record<string, any> = {};
        for (const prop of node.properties) {
            const key = prop.key.name;
            const value = literalFromNode(prop.value);
            obj[key] = value;
        }
        return obj;
    }

    return undefined;
}

```

5. Historia czatu (lib/chat-history.ts)

Odpowiedzialność: Persistencja danych konwersacji w LocalStorage

Interfejs Chat:

```

export interface Chat {
    id: string; // Unikalny identyfikator
    title: string; // Tytuł wygenerowany z pierwszej wiadomości
}

```

```

    messages: Message[]; // Pełna historia wiadomości
    createdAt: number; // Timestamp utworzenia
    updatedAt: number; // Timestamp ostatniej aktualizacji
  }

```

API Funkcje:

1. **loadChatHistory()**: Ładuje wszystkie czaty z localStorage

```

export function loadChatHistory(): Chat[] {
  const stored = localStorage.getItem(STORAGE_KEY);
  return stored ? JSON.parse(stored) : [];
}

```

2. **saveChat(chat)**: Zapisuje lub aktualizuje pojedynczy czat

```

export function saveChat(chat: Chat): void {
  const chats = loadChatHistory();
  const existingIndex = chats.findIndex((c) => c.id === chat.id);

  if (existingIndex >= 0) {
    chats[existingIndex] = { ...chat, updatedAt: Date.now() };
  } else {
    chats.unshift(chat); // Nowy czat na początek
  }

  saveChatHistory(chats);
}

```

3. **deleteChat(chatId)**: Usuwa czat z historii

```

export function deleteChat(chatId: string): void {
  const chats = loadChatHistory();
  saveChatHistory(chats.filter((c) => c.id !== chatId));
}

```

4. **getChatTitle(messages)**: Generuje tytuł z pierwszej wiadomości

```

export function getChatTitle(messages: Message[]): string {
  const firstUserMessage = messages.find((m) => m.role === "user");
  const content = firstUserMessage?.content || "Nowa konwersacja";
  return content.length > 50 ? content.substring(0, 50) + "... " : content;
}

```

Przepływ Danych

Scenariusz 1: Wysłanie Wiadomości

1. Użytkownik wpisuje wiadomość w ChatInterface

↓

2. `handleSendMessage()`
- Tworzy Message object z role: "user"
- `setMessages([...prev, newMessage])`
- Dodaje loading message

↓

3. `fetch("/api/chat", { messages })`
→ POST request do API route

↓

4. API Route (`route.ts`)
- Waliduje API key
- Buduje `groqMessages` z system prompt
- `fetch("https://api.groq.com/...")`

↓

5. Groq API (Llama 4 Maverick)
- Przetwarza prompt
- Generuje odpowiedź z kodem JSX
- Zwraca completion

↓

6. API Route - Parsowanie
- Ekstrahuje JSX z code block
- `parse()` → AST
- `jsxToTree()` → `ComponentTree`
- `return { message, tree }`

↓

7. ChatInterface - Aktualizacja
- Zastępuje loading message odpowiedzią
- `setMessages()` z message + tree

↓

8. `page.tsx` - Auto-save
- `useEffect` wykrywa zmianę messages

- saveChat() → localStorage
- Aktualizuje chatHistory

↓

9. PreviewPanel - Renderowanie
- Znajduje ostatnią wiadomość assistant z tree
 - renderNode(tree) → React elements
 - Wyświetla podgląd na żywo

Scenariusz 2: Ładowanie Historycznego Czatu

Użytkownik wybiera chat in HistoryModal

↓

handleLoadChat(chat)

↓

setMessages(chat.messages)

setCurrentChatId(chat.id)

↓

ChatInterface zostaje wyrenderowany z załadowanymi wiadomościami

↓

PreviewPanel wyświetla wygenerowany interfejs

Konfiguracja i Środowisko

Zmienne Środowiskowe

.env.local (wymagane):

GROQ_API_KEY=gsk_... *# Klucz API do Groq (wymagany)*

Tailwind CSS Config

tailwind.config.mjs:

```
export default {
  content: [
    "./pages/**/*.ts,tsx",
    "./components/**/*.ts,tsx",
    "./app/**/*.ts,tsx",
  ],
  theme: {
    extend: {
      colors: {
        // Zmienne CSS dla theming

```

```

        background: "hsl(var(--background))",
        foreground: "hsl(var(--foreground))",
        primary: { DEFAULT: "hsl(var(--primary))", foreground: "..."},
        // ... pozostałe kolory
    },
    borderRadius: {
        lg: "var(--radius)",
        md: "calc(var(--radius) - 2px)",
        sm: "calc(var(--radius) - 4px)",
    },
    animation: {
        "accordion-down": "accordion-down 0.2s ease-out",
        "accordion-up": "accordion-up 0.2s ease-out",
    },
},
},
plugins: [require("tailwindcss-animate")],
};

```

TypeScript - konfiguracja

tsconfig.json:

```

{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "jsx": "preserve",
    "module": "esnext",
    "moduleResolution": "bundler",
    "paths": {
      "@/*": ["./*"] // Path alias dla importów
    },
    "strict": true,
    "noEmit": true,
    "incremental": true
  }
}

```

Build & Development Scripts

```

npm run dev           # Uruchamia Next.js dev server na port 3000

npm run build         # Kompiluje aplikację do .next/
npm run start         # Uruchamia production server

npm run lint          # ESLint

```

Integracje Zewnętrzne

1. Groq API (LLM)

Endpoint: `https://api.groq.com/openai/v1/chat/completions`

Model: `meta-llama/llama-4-maverick-17b-128e-instruct`

Format zapytania:

```
{
  model: "meta-llama/llama-4-maverick-17b-128e-instruct",
  messages: [
    { role: "system", content: "..." },
    { role: "user", content: "..." },
    { role: "assistant", content: "..." }
  ],
  max_tokens: 4096,
  temperature: 0.7
}
```

Format odpowiedzi:

```
{
  choices: [{
    message: {
      role: "assistant",
      content: "Explanation...\n```tsx\n<Component />\n```"
    },
    finish_reason: "stop"
  }],
  usage: { prompt_tokens: 500, completion_tokens: 1200 }
}
```

Rate Limit:

- Zależne od planu Groq oraz wybranego modelu

Błędy:

```
try {
  const response = await fetch(GROQ_API_URL, options);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
} catch (error) {
  console.error("Groq API Error:", error);
  return NextResponse.json(
    {
      error: "Failed to get response from LLM",
    }
  );
}
```

```

    },
    { status: 500 }
  );
}

```

2. Web Speech API

Dostępność:

```

const isSpeechRecognitionAvailable =
  "SpeechRecognition" in window || "webkitSpeechRecognition" in window;

```

Konfiguracja:

```

recognition.continuous = false; // Pojedynczy wynik
recognition.interimResults = false; // Tylko wynik finalny
recognition.lang = "pl-PL"; // Polski język

```

Browser Support:

- Chrome/Edge: Pełne wsparcie
- Safari: Wsparcie z prefixem webkit

3. LocalStorage API

Storage Key: "chat-history"

Data Structure:

```

localStorage.getItem("chat-history") → string (JSON)

```

```

Parsed structure: Chat[] = [
  {
    id: "chat-1704188400000",
    title: "Stwórz formularz logowania",
    messages: [...],
    createdAt: 1704188400000,
    updatedAt: 1704188500000
  },
  // ... więcej czatów
]

```

Storage Limits:

- Limit: ~5-10MB (zależnie od przeglądarki)
- Monitoring: Implementacja try-catch dla quota exceeded errors

Bezpieczeństwo

1. API Key Protection

Server-Side Only:

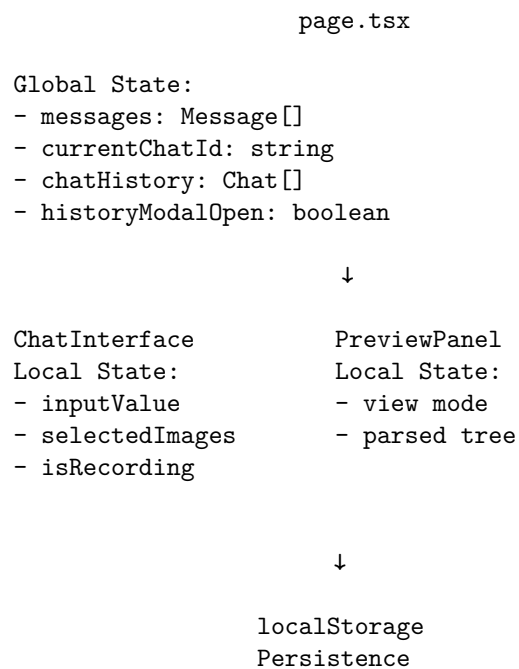
```
// Bezpieczne: API key tylko w API route
if (!process.env.GROQ_API_KEY) {
  return NextResponse.json({ error: "..." }, { status: 500 });
}
```

2. Input Sanitization

- React automatycznie escapuje tekst w JSX
-

Zarządzanie Stanem

State Architecture Diagram



State Update Patterns

1. Immutable Updates:

```
setMessages((prev) => [...prev, newMessage]);
```

2. Functional Updates:

```
// Gdy nowy stan zależy od poprzedniego  
setMessages((prevMessages) =>  
  prevMessages.map((msg) => (msg.id === loadingId ? assistantMessage : msg))  
);
```

3. Batched Updates:

```
// React 18+ automatycznie batchuje  
setMessages([...prev, userMessage]);  
setSelectedImages([]);  
setInputValue("");  
// ^ Wszystkie w jednym re-render
```

Dokumentacja Zewnętrzna

- **Next.js:** <https://nextjs.org/docs>
- **shadcn/ui:** <https://ui.shadcn.com>
- **Groq API:** <https://console.groq.com/docs>
- **Tailwind CSS:** <https://tailwindcss.com/docs>